

myAssistente5

Personal Desktop Assistant – Documentation Toolkit

(4) Memory Configuration Manual · Release 5 · English

[Home](#)[Author](#)[GB English](#)[IT Italiano](#)

Contents: [The memory.json file](#) Store a: [data entry](#) [folder link](#) [website](#)
[program](#) [website](#) [anything](#) [Documentation](#)

(4) Memory Manual

- **Purpose:** This manual explains how to configure the myAssistente memory system to make the most of its capabilities.
- **Prerequisites:** The myAssistente executable for your operating system, or alternatively the Python files `myAssistente.py` and (optional, for AIML support) `aiml_parser.py`, with Python installed on your machine (multiplatform).
- **References:** (1) *Installation Manual*, (2) *User Manual*, (3) *Configuration Manual*.

All command names can be customized in the `lang_XX.json` file, and may therefore differ from those shown in this manual. For details, refer to the (3) *Configuration Manual*.



File locations

The file described in this manual is located inside the `_dati/` folder.

- **`_dati/memory.json`:** Stored data.

The memory.json file

Note: If the `memory.json` file is missing, the system will notify the user and create a new empty file when the first data entry is added through the interface. If the file exists but is corrupted, it will be renamed to `memory.json.bak` to preserve any valid data, and a new file will be created.

Data can be added directly from the myAssistente interface using the commands `remember` and `learn`. See the *(2) User Manual* for instructions.

You may also modify or add data manually. Open the file with a text editor:

Store a data entry

When you write a value inside the `"dati"` field, the system will automatically recognize it. If the JSON file is invalid, the assistant will notify you; otherwise, you can insert data directly through the interface.

```
{
  "nome": "Codice",
  "alias": "tasse",
  "soggetto": "Emanuele",
  "dati": "CDFTSS99T30L424X",
  "tag": "codici",
  "avatar": "magazziniere"
},
```

`nome` — required field, the term used to request the data.

`alias` — optional, alternative terms to request the same data.

`soggetto` — optional, indicates the owner of the data, useful when storing information for multiple people.

`dati` — required, contains the actual value.

`tag` — optional, useful for searching: asking `search codici` will return all entries with that tag.

`avatar` — optional, specifies which image the assistant should display when showing the data. If omitted, the system selects a random image based on the configuration in `config.json`.

Command to retrieve the information: `find [name]` or `tell me [name]`.

Store a folder link

When you write a link inside the `"dati"` field, the system will automatically recognize it.

```
{
  "nome": "Python",
  "alias": "progetti",
  "soggetto": "open a folder",
  "dati": "C:\\Python\\Progetti\\",
  "tag": "folders",
  "avatar": "informatica"
},
```

JSON requires double backslashes, as in `C:\\Python\\Progetti\\`, but the assistant will display and open the path correctly.

To open the folder: `open [name]`.

Store a website

You can also store a website link in the same way.

```
{
  "nome": "Autore",
  "alias": "Cassani",
  "soggetto": "Emanuele",
  "dati": "https://www.steppa.net/cassani/index.htm",
  "tag": "web",
  "avatar": "agenda"
},
```

Standard slashes / do not require escaping. Simply write the URL and the assistant will open it in the browser.

To open the website: `open [name]` .

Store a program

You can also store a program to be executed.

```
{
  "nome": "browser",
  "alias": "web",
  "soggetto": "brave",
  "dati": "C:\\Program Files\\BraveSoftware\\Brave-Browser\\Appli
  "tag": "programmi",
  "avatar": "bibliotecaria"
},
```

Enter the executable path inside the `dati` field. The same method applies to programs, folders, and files.

Command to launch the program: `open [name]` .

Store anything

Feel free to experiment with anything that comes to mind: the assistant is extremely flexible.

```
{
  "nome": "shell",
  "alias": "prompt",
  "soggetto": "Windows11",
  "dati": "cmd.exe",
  "tag": "comandi",
  "avatar": "informatica"
},
```

Example of a Windows command that opens the command prompt.

Command to run it: `open shell`.

Documentation

Available documentation for **myAssistente5**.

The documents are also available in the [/DOCS](#) folder of the repository. For missing documentation, refer to previous English or Italian releases.

- **(1) Installation Manual** — installing the program for Windows, Linux, or multiplatform
- **(2) User Manual** — myAssistente user interface
- **(3) Configuration Manual** — `config.json` file, advanced settings
- **(4) Memory Manual** — `memory.json` file, advanced settings
- **(5) Language Manual** — `lang[.json]` files, advanced customization
- **(6) AIML Manual** — `aiml/[LANG]/[filename].aiml` files

- **(7) Advanced AIML Manual** — `aiml/IT/testConiglio.aiml` file

Toolkit **myAssistente5** — Personal desktop assistant

[Author's website](#) · © 2026 [Emanuele Cassani](#)